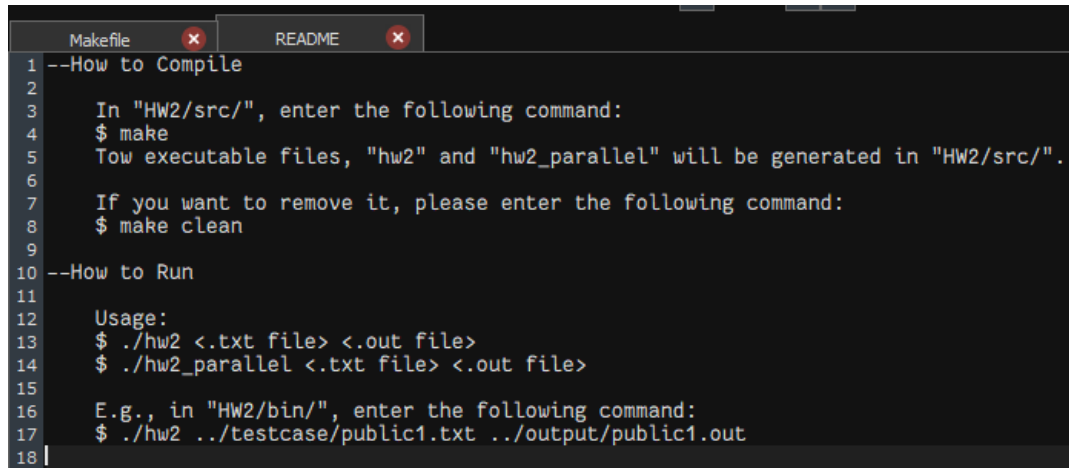


1. Your name and student ID

110062222 李子賢

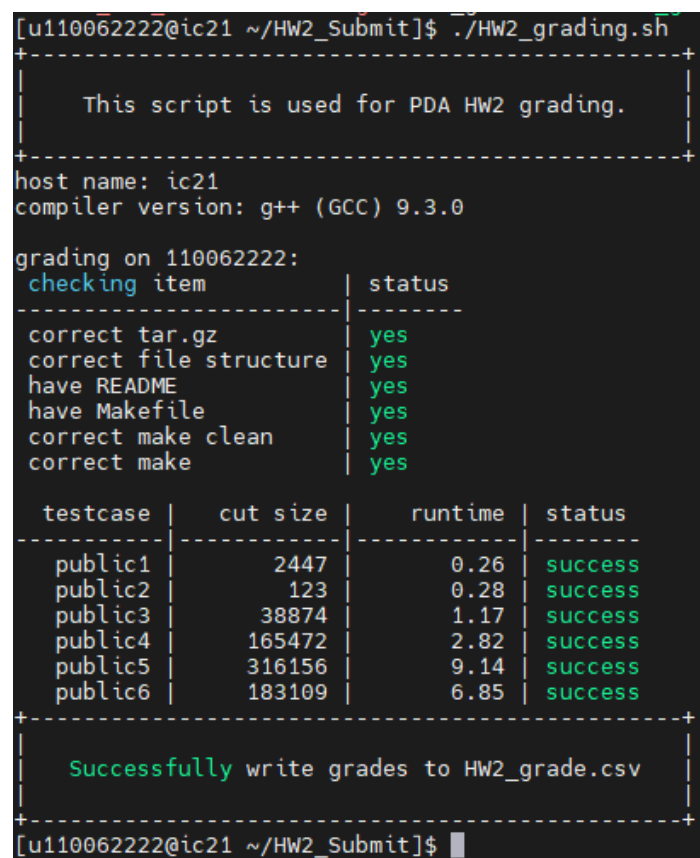
2. How to compile and execute your program, and give an execution example.



The screenshot shows a terminal window with two tabs: 'Makefile' and 'README'. The 'README' tab is active, displaying the following text:

```
1 --How to Compile
2
3 In "HW2/src/", enter the following command:
4 $ make
5 Two executable files, "hw2" and "hw2_parallel" will be generated in "HW2/src/".
6
7 If you want to remove it, please enter the following command:
8 $ make clean
9
10 --How to Run
11
12 Usage:
13 $ ./hw2 <.txt file> <.out file>
14 $ ./hw2_parallel <.txt file> <.out file>
15
16 E.g., in "HW2/bin/", enter the following command:
17 $ ./hw2 ../testcase/public1.txt ../output/public1.out
18
```

3. The final cut size and the runtime of each testcase. Paste the screenshot of the result of running the HW2_grading.sh as the picture shown below.



The screenshot shows a terminal window with the command `./HW2_grading.sh` executed. The output is as follows:

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh
+-----+
| This script is used for PDA HW2 grading. |
+-----+
host name: ic21
compiler version: g++ (GCC) 9.3.0

grading on 110062222:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes

testcase | cut size | runtime | status
+-----+
public1 | 2447 | 0.26 | success
public2 | 123 | 0.28 | success
public3 | 38874 | 1.17 | success
public4 | 165472 | 2.82 | success
public5 | 316156 | 9.14 | success
public6 | 183109 | 6.85 | success
+-----+

Successfully write grades to HW2_grade.csv
[u110062222@ic21 ~/HW2_Submit]$
```

4. The details of your algorithm. You could use flow chart(s) and/or pseudo code to help elaborate your algorithm. If your algorithm is similar to some previous work, please cite the corresponding paper(s) and reveal your difference(s).

整體架構是根據 FM 演算法去修改。

不同之處

1. 在講義中，取得 max gain cell 的時候是依靠 bucket list，因為 edge weight 都是 1，bucket list 大小最大只會到 $2 \times \text{Pin}$ 的最大數量，但作業中的 edge weight 不為 1，因此我是用一個 multiset 來存所有 Cell，根據當前 Cell Gain 的值自動排序。
2. 上課中老師是說 initial partition 可以在面積限制之下隨機分配，或是先填滿 Die A，再填 Die B。但在作業中，這兩種方法都會造成面積超出限制。由於作業的每個 Cell 在不同 Die 的大小不同，我猜可能有出現在 Die A 面積遠比 Die B 大的情況，導致超出限制。因此我採用的方法是，對於每個 Cell，根據 $\text{Min}(\text{Area_in_A}, \text{Area_in_B})$ 的大小，由小排到大，再優先取較小的面積去放置，就能符合作業的面積限制。

在計算 Pass 的時候，我會開兩個一模一樣的資料結構。

```
struct Net {
    Net() {}
    Net(long long weight) : weight(weight) {}
    void update_test() {
        cells_in_A_test = cells_in_A;
        cells_in_B_test = cells_in_B;
    }
    // 會變的
    unordered_map<string, Cell*> cells_in_A, cells_in_B;
    unordered_map<string, Cell*> cells_in_A_test, cells_in_B_test;
    // 不會變的
    long long weight = 0;
    unordered_map<string, Cell*> cells;
    long long max_area_in_A = 0, max_area_in_B = 0;
};

struct Cell{
    Cell() {}
    Cell(long long area_in_A, long long area_in_B, string cell_name) : area_in_A(area_in_A), area_in_B(area_in_B), cell_name(cell_name) {}
    void update_test() {
        die_test = die;
        gain_test = gain;
    }
    // 會變的
    string die, die_test;
    long long gain = 0, gain_test = 0;
    bool is_locked = false;
    // 不會變的
    long long area_in_A = 0, area_in_B = 0;
    long long connected_weight = 0;
    string cell_name;
    vector<Net*> nets;
};
```

先用名稱為 _test 的資料結構 (cells_in_A_test、cells_in_B_test、die_test、gain_test 等等) 模擬 Pass 一遍，在符合面積限制的情況下，把所有移動的 Cell、得到的 Gain 都存到 Vector 中，跑完後再取出 Gain Vector 的 Max

Partial Gain Sum，取出前 K 個真的要移動的 Cell，真的執行結果存到另一個資料結構中 (cells_in_A、cells_in_B、die、gain 等等)，再複製給 _test，持續迭代上述操作直到 Max Partial Gain Sum ≤ 0 。

5. What techniques did you use to improve your solution's quality?

Additionally, analyze how they contributed to the improvements. Plot the effects of those different settings like the ones shown below.

照著上面的方法做能夠通過 Baseline，但是 Cutsizes 不夠低，因此我在 Initial Partition、Max Gain Cell (前面提到的 Multiset) 的排序邏輯中，加入了一些 Cell 相關的資訊，包含：Cell 連到的 Net 數量大小、最小的面積大小、連接到的 Nets 的總 Weight 大小等等。

```
sort(cells_vector.begin(), cells_vector.end(), [](Cell* a, Cell* b) {
    long long min_area_a = min(a->area_in_A, a->area_in_B);
    long long min_area_b = min(b->area_in_A, b->area_in_B);
    if (min_area_a == min_area_b) {
        if (a->connected_weight == b->connected_weight) {
            if (a->nets.size() == b->nets.size()) {
                if (a->area_in_A != b->area_in_A) {
                    return a->area_in_A < b->area_in_A;
                } else {
                    return a->area_in_B < b->area_in_B;
                }
            }
            return a->nets.size() > b->nets.size();
        }
        return a->connected_weight > b->connected_weight;
    }
    return min_area_a < min_area_b;
});
```

像是在 Initial Partition 中，優先考慮面積大小才能符合面積限制，但是 Cell 數量遠大於 LibCell 數量，因此有很多 Cell 面積都一樣大，這時候可以多考慮 Cell 連接到的 Net 總 Weight 值，優先放 Weight 大的 Cell，如果 Weight 也一樣，再考慮到 Cell 連接到的 Net 數量 (Net Size)，更有機會讓它們在同一邊。如果都一樣，再去細部比較兩個 Cell 分別在 A、B Die 的面積大小，優先放更小的那個。

```
struct CompareCell {
    bool operator()(Cell* a, Cell* b) {
        if (a->gain_test == b->gain_test) {
            if (a->nets.size() == b->nets.size()) {
                long long min_area_a = min(a->area_in_A, a->area_in_B);
                long long min_area_b = min(b->area_in_A, b->area_in_B);
                if (min_area_a == min_area_b) {
                    if (a->area_in_A != b->area_in_A) {
                        return a->area_in_A < b->area_in_A;
                    } else {
                        return a->area_in_B < b->area_in_B;
                    }
                }
                return min_area_a < min_area_b;
            }
            return a->nets.size() > b->nets.size();
        }
        return a->gain_test > b->gain_test;
    }
};
multiset<Cell*, CompareCell> cells_set;
```

在計算 Max Gain 時的 Multiset 的排序邏輯中，Cell Gain 的大小是最重要的，但也會發生許多 Cell Gain 相同的情況，這時候可以優先移動 Net Size 大的 Cell，可能會讓 Solution Space 更大，結果更彈性。如果 Net Size 也相同，就優先取面積更小的 Cell，比較不會受到面積限制而影響結果。

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh

This script is used for PDA HW2 grading.

host name: ic21
compiler version: g++ (GCC) 9.3.0

grading on 110062222:
checking item | status
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes

testcase | cut size | runtime | status
public1 | 7865 | 0.22 | success
public2 | 490 | 0.24 | success
public3 | 55656 | 1.37 | success
public4 | 252646 | 2.13 | success
public5 | 469113 | 5.08 | success
public6 | 255424 | 6.20 | success

Successfully write grades to HW2_grade.csv
[u110062222@ic21 ~/HW2_Submit]$
```

優化前

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh

This script is used for PDA HW2 grading.

host name: ic21
compiler version: g++ (GCC) 9.3.0

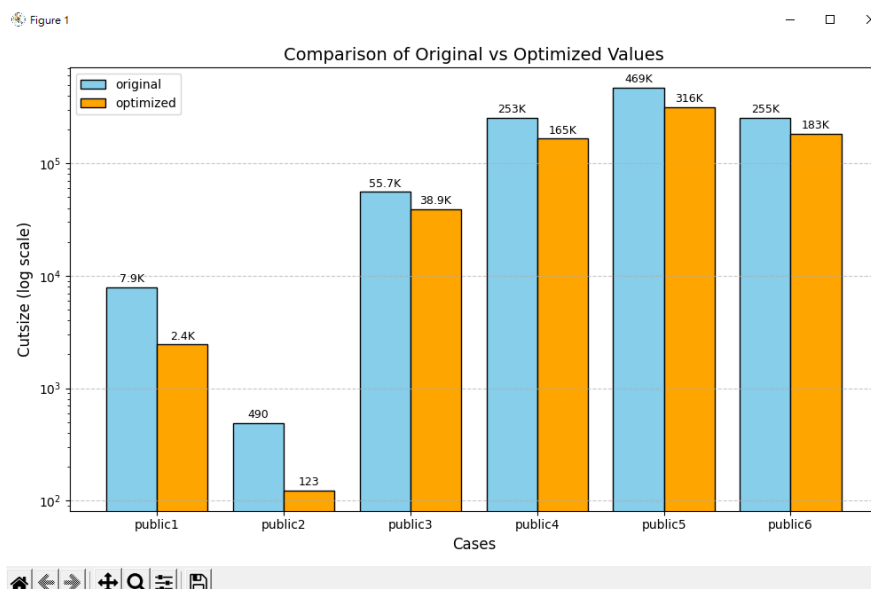
grading on 110062222:
checking item | status
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes

testcase | cut size | runtime | status
public1 | 2447 | 0.26 | success
public2 | 123 | 0.28 | success
public3 | 38874 | 1.17 | success
public4 | 165472 | 2.82 | success
public5 | 316156 | 9.14 | success
public6 | 183109 | 6.85 | success

Successfully write grades to HW2_grade.csv
[u110062222@ic21 ~/HW2_Submit]$
```

優化後

因為我的 Runtime 都很低，這邊就不比較 Runtime 的部分，著重在減少 Cutsizes 的部分。



可以看出整體的 Cutsizes 都有大幅改善。

6. If you implement parallelization (for algorithm itself), please describe the implementation details and provide some experimental results.

在 FM 演算法的核心邏輯中，Bottleneck 主要在多次迭代 Pass 的計算中，但是 Pass 的計算是每次取出 Gain 最大的 Cell 去移動，會有很高的 Dependency，我認為不太能做平行化。

在我實作的演算法中，我認為能夠平行化的部分只有計算 Initial Cell Gain、Initial Cutsizes 和 Pass 完複製資料的時候。

其中我認為對 Initial 計算做平行化不會有太大的影響，畢竟只會跑一次，但 Pass 完複製資料我認為有可能會造成更大的影響。前面提到，在 Pass 時我會開兩個資料結構去模擬及執行，每次 Pass 前要把兩者變成一致，這時候就要把上一輪執行後資料複製到模擬用的資料。

```
void update_test_values(Die* die, vector<Net*>& nets_vector, unordered_map<string, Cell*>& cells_map) {
    for (int i = 0; i < NUM_OF_DIE; i++) die[i].update_test();
    for (auto net : nets_vector) net->update_test();
    for (auto cell : cells_map) cell.second->update_test();
}
```

```
struct Net {
    Net() {}
    Net(long long weight) : weight(weight) {}
    void update_test() {
        cells_in_A_test = cells_in_A;
        cells_in_B_test = cells_in_B;
    }
    // 會變的
    unordered_map<string, Cell*> cells_in_A, cells_in_B;
    unordered_map<string, Cell*> cells_in_A_test, cells_in_B_test;
    // 不會變的
    long long weight = 0;
    unordered_map<string, Cell*> cells;
    long long max_area_in_A = 0, max_area_in_B = 0;
};
```

```
struct Cell {
    Cell() {}
    Cell(long long area_in_A, long long area_in_B, string cell_name) : area_in_A(area_in_A), area_in_B(area_in_B), cell_name(cell_name) {}
    void update_test() {
        die_test = die;
        gain_test = gain;
    }
    // 會變的
    string die, die_test;
    long long gain = 0, gain_test = 0;
    bool is_locked = false;
    // 不會變的
    long long area_in_A = 0, area_in_B = 0;
    long long connected_weight = 0;
    string cell_name;
    vector<Net*> nets;
};
```

由於 Pass 要執行多次，每次都要複製大量的資料過去另一邊，差不多是複製兩倍 Cell 的數量，因此我決定在這邊做平行化。

平行化的部分我是用 openmp , ex: #pragma omp parallel for num_threads(64)

```
void update_test_values(Die* die, vector<Net*>& nets_vector, unordered_map<string, Cell*>& cells_map) {
    for (int i = 0; i < NUM_OF_DIE; i++) {
        die[i].update_test();
    }

    #pragma omp parallel for num_threads(64)
    for (size_t i = 0; i < nets_vector.size(); i++) {
        nets_vector[i]->update_test();
    }

    vector<pair<string, Cell*>> cells_vec(cells_map.begin(), cells_map.end());
    #pragma omp parallel for num_threads(64)
    for (size_t i = 0; i < cells_vec.size(); i++) {
        cells_vec[i].second->update_test();
    }
}
```

實驗結果

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh
+-----+
| This script is used for PDA HW2 grading. |
+-----+
host name: ic21
compiler version: g++ (GCC) 7.3.0

grading on 110062222:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes

testcase | cut size | runtime | status
+-----+
public1 | 2447 | 0.27 | success
public2 | 123 | 0.27 | success
public3 | 38874 | 1.18 | success
public4 | 165472 | 2.99 | success
public5 | 316156 | 9.41 | success
public6 | 183109 | 6.95 | success

+-----+
| Successfully write grades to HW2_grade.csv |
+-----+
[u110062222@ic21 ~/HW2_Submit]$
```

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh
+-----+
| This script is used for PDA HW2 grading. |
+-----+
host name: ic21
compiler version: g++ (GCC) 7.3.0

grading on 110062222:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes

testcase | cut size | runtime | status
+-----+
public1 | 2447 | 0.36 | success
public2 | 123 | 0.33 | success
public3 | 38874 | 1.37 | success
public4 | 165472 | 3.13 | success
public5 | 316156 | 11.84 | success
public6 | 183109 | 7.45 | success

+-----+
| Successfully write grades to HW2_grade.csv |
+-----+
[u110062222@ic21 ~/HW2_Submit]$
```

Core 1

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh
+-----+
| This script is used for PDA HW2 grading. |
+-----+
host name: ic21
compiler version: g++ (GCC) 7.3.0

grading on 110062222:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes

testcase | cut size | runtime | status
+-----+
public1 | 2447 | 0.35 | success
public2 | 123 | 0.32 | success
public3 | 38874 | 1.39 | success
public4 | 165472 | 3.22 | success
public5 | 316156 | 12.23 | success
public6 | 183109 | 7.48 | success

+-----+
| Successfully write grades to HW2_grade.csv |
+-----+
[u110062222@ic21 ~/HW2_Submit]$
```

Core 4

Core 2

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh
+-----+
| This script is used for PDA HW2 grading. |
+-----+
host name: ic21
compiler version: g++ (GCC) 7.3.0

grading on 110062222:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes

testcase | cut size | runtime | status
+-----+
public1 | 2447 | 0.35 | success
public2 | 123 | 0.31 | success
public3 | 38874 | 1.32 | success
public4 | 165472 | 3.03 | success
public5 | 316156 | 12.37 | success
public6 | 183109 | 7.49 | success

+-----+
| Successfully write grades to HW2_grade.csv |
+-----+
[u110062222@ic21 ~/HW2_Submit]$
```

Core 8

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh
+-----+
| This script is used for PDA HW2 grading. |
+-----+
host name: ic21
compiler version: g++ (GCC) 7.3.0
grading on 110062222:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes
+-----+
testcase | cut size | runtime | status
+-----+
public1 | 2447 | 0.35 | success
public2 | 123 | 0.31 | success
public3 | 38874 | 1.45 | success
public4 | 165472 | 3.08 | success
public5 | 316156 | 12.65 | success
public6 | 183109 | 7.82 | success
+-----+
Successfully write grades to HW2_grade.csv
+-----+
[u110062222@ic21 ~/HW2_Submit]$
```

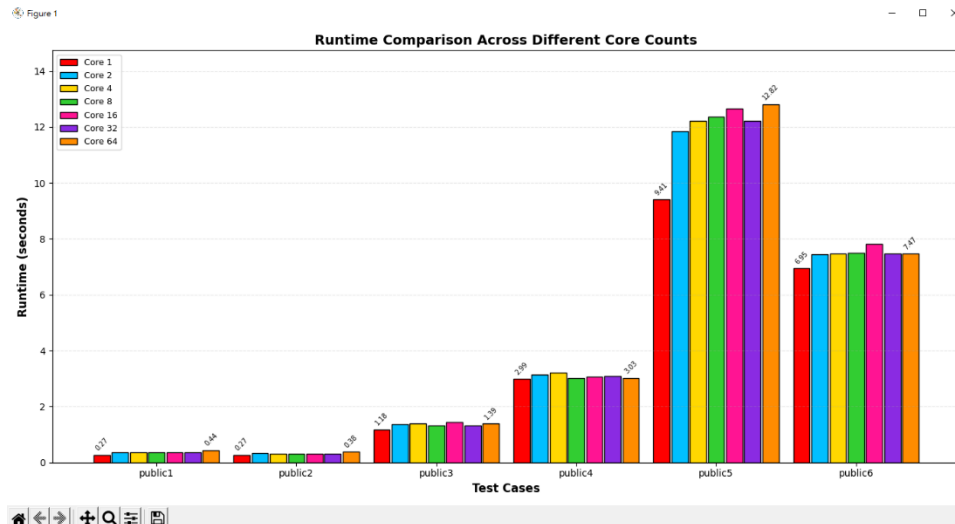
Core 16

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh
+-----+
| This script is used for PDA HW2 grading. |
+-----+
host name: ic21
compiler version: g++ (GCC) 7.3.0
grading on 110062222:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes
+-----+
testcase | cut size | runtime | status
+-----+
public1 | 2447 | 0.35 | success
public2 | 123 | 0.31 | success
public3 | 38874 | 1.32 | success
public4 | 165472 | 3.09 | success
public5 | 316156 | 12.22 | success
public6 | 183109 | 7.47 | success
+-----+
Successfully write grades to HW2_grade.csv
+-----+
[u110062222@ic21 ~/HW2_Submit]$
```

Core 32

```
[u110062222@ic21 ~/HW2_Submit]$ ./HW2_grading.sh
+-----+
| This script is used for PDA HW2 grading. |
+-----+
host name: ic21
compiler version: g++ (GCC) 7.3.0
grading on 110062222:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes
+-----+
testcase | cut size | runtime | status
+-----+
public1 | 2447 | 0.44 | success
public2 | 123 | 0.38 | success
public3 | 38874 | 1.39 | success
public4 | 165472 | 3.03 | success
public5 | 316156 | 12.82 | success
public6 | 183109 | 7.47 | success
+-----+
Successfully write grades to HW2_grade.csv
+-----+
[u110062222@ic21 ~/HW2_Submit]$
```

Core 64



從上圖可以看出平行化後整體 Runtime 沒有太大的變化。值得注意的是當 Core 從 1 變 2 時，Runtime 反而有明顯的增加，單核心的 Performance 反而比較好，猜測是因為資料不夠大或平行化沒做好，無法利用多核心的優勢。我認為我實作的演算法不適合做平行化加速，單核心的 Runtime 不長之外，FM 演算法內部的計算含有大量的 Dependency，能利用平行化優勢的部分不多。

7. What have you learned from this homework? What problem(s) have you encountered in this homework?

這個作業幫我複習大量的 C++ 操作，從零開始實現講義的演算法還滿有趣的，雖然花很多時間，但能做出來真是太好了。但我不知道為什麼我的 Runtime 跟同學比起來似乎特別低，不確定差在哪，希望在 Private Case 也能順利。

我遇到最大的問題是在 Initial Partition 的部分，Initial Partition 會嚴重影響 Cutszie 的結果，一開始我試過很多方法，像是隨機分配多次迭代取最好的結果、把相同的 Net 優先放在一起等等，雖然效果很不錯，但不管哪種方法都會違反面積限制，最後只好妥協先放面積小的 Cell，再從這邊想要怎麼優化。